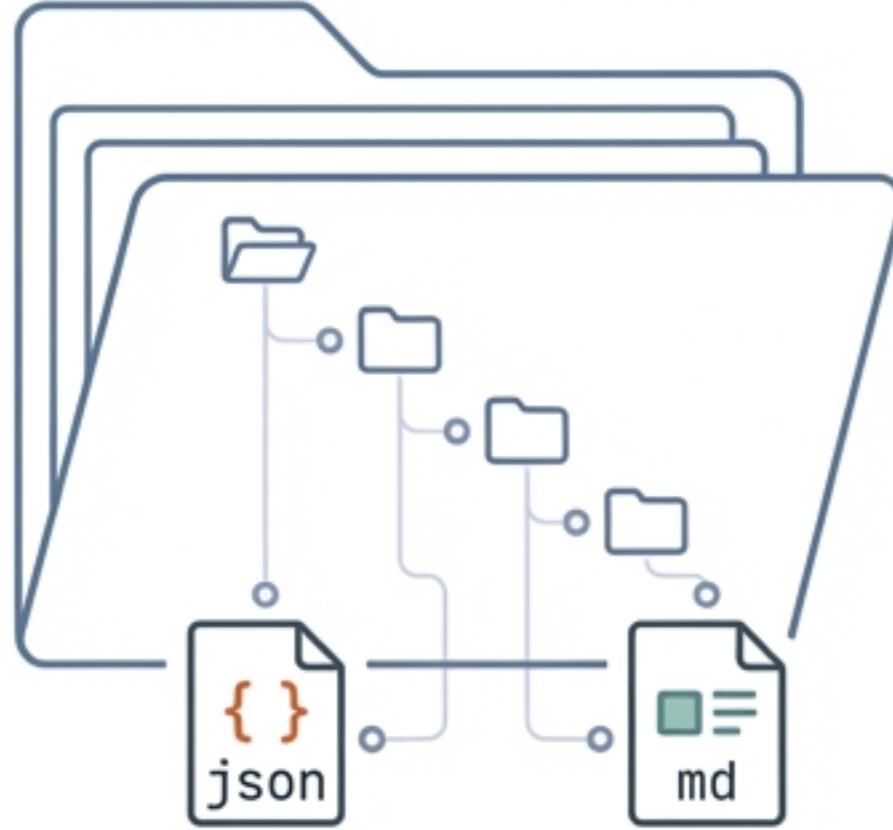




Issue Tracking & Version Documentation System

A File-System Based Approach for Human-AI Collaboration

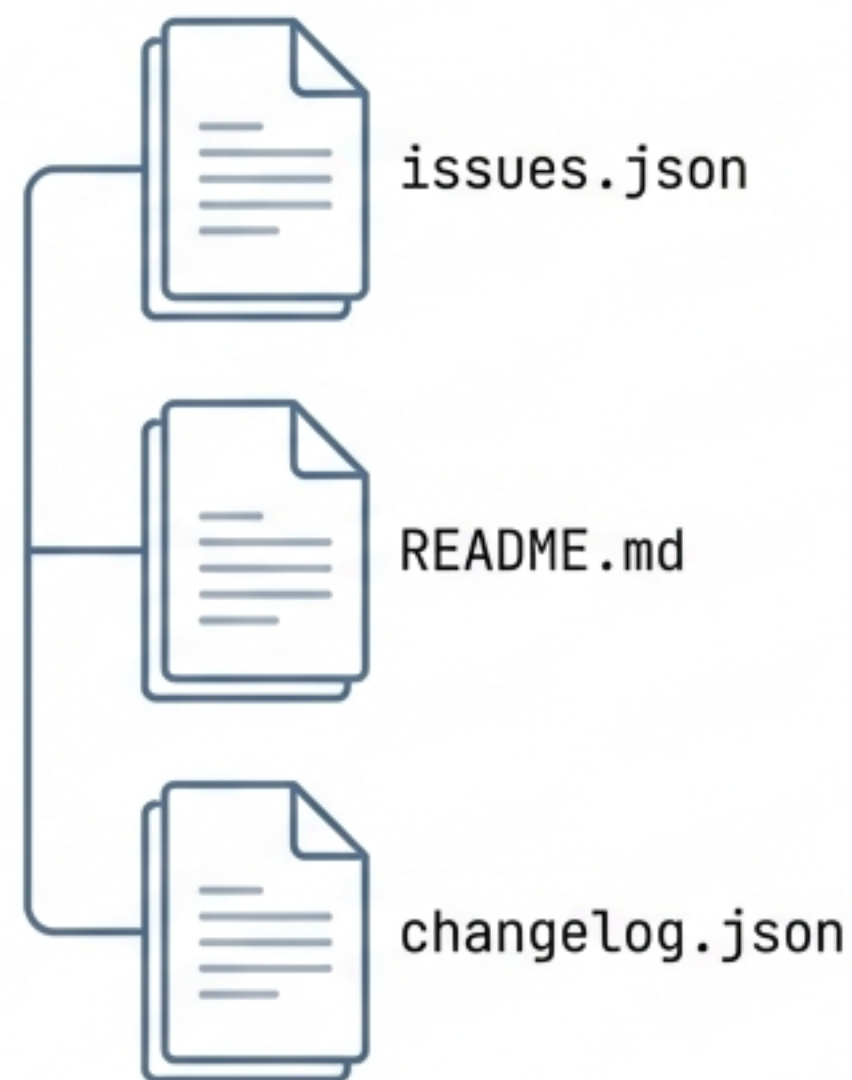
Integrating Structured Logic into the UI

The Problem: Ephemeral Context



Context is lost in chat history.
Logic is transient.

The Solution: Persistent State



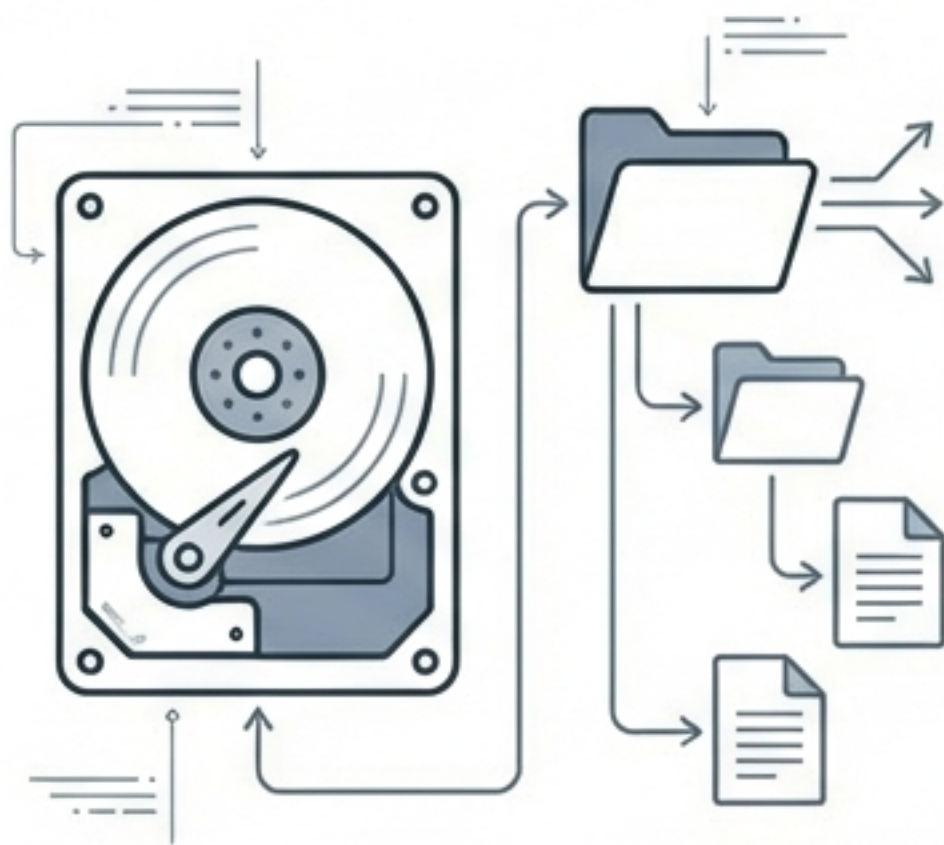
Context is stored in the File System.
Logic is permanent.

Key Value Propositions

1. Replaces ephemeral chat context with persistent artifacts.
2. Utilizes the file system as the database (No SQL).
3. Embeds a “Mini GitHub” directly into the Graph Transformation UI.

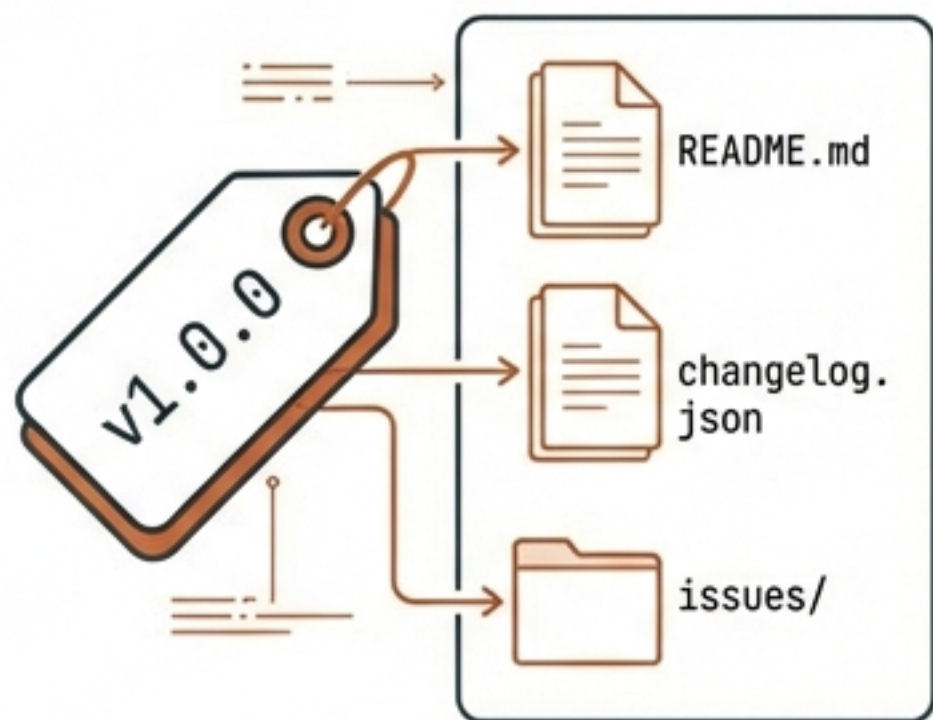
Core Architecture Philosophy

File System as Database



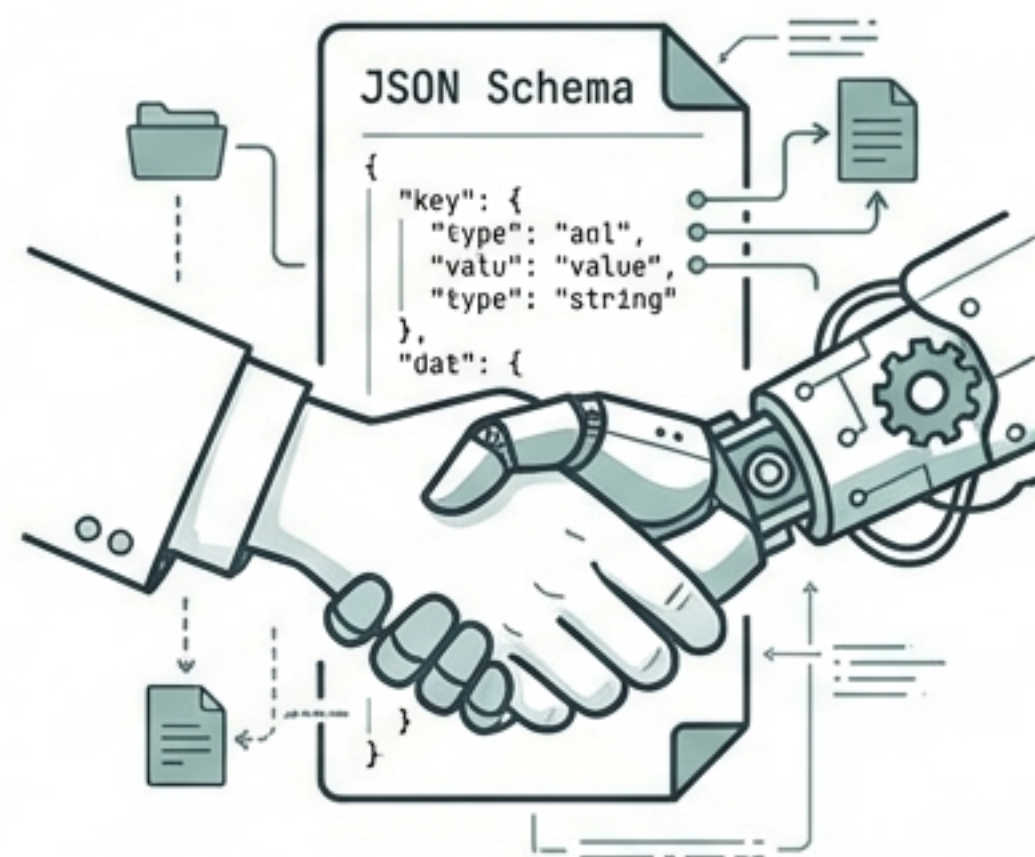
No external SQL/NoSQL required.
All state is stored in JSON files
read via GET requests from the
existing FastAPI backend.

Version-Centricity



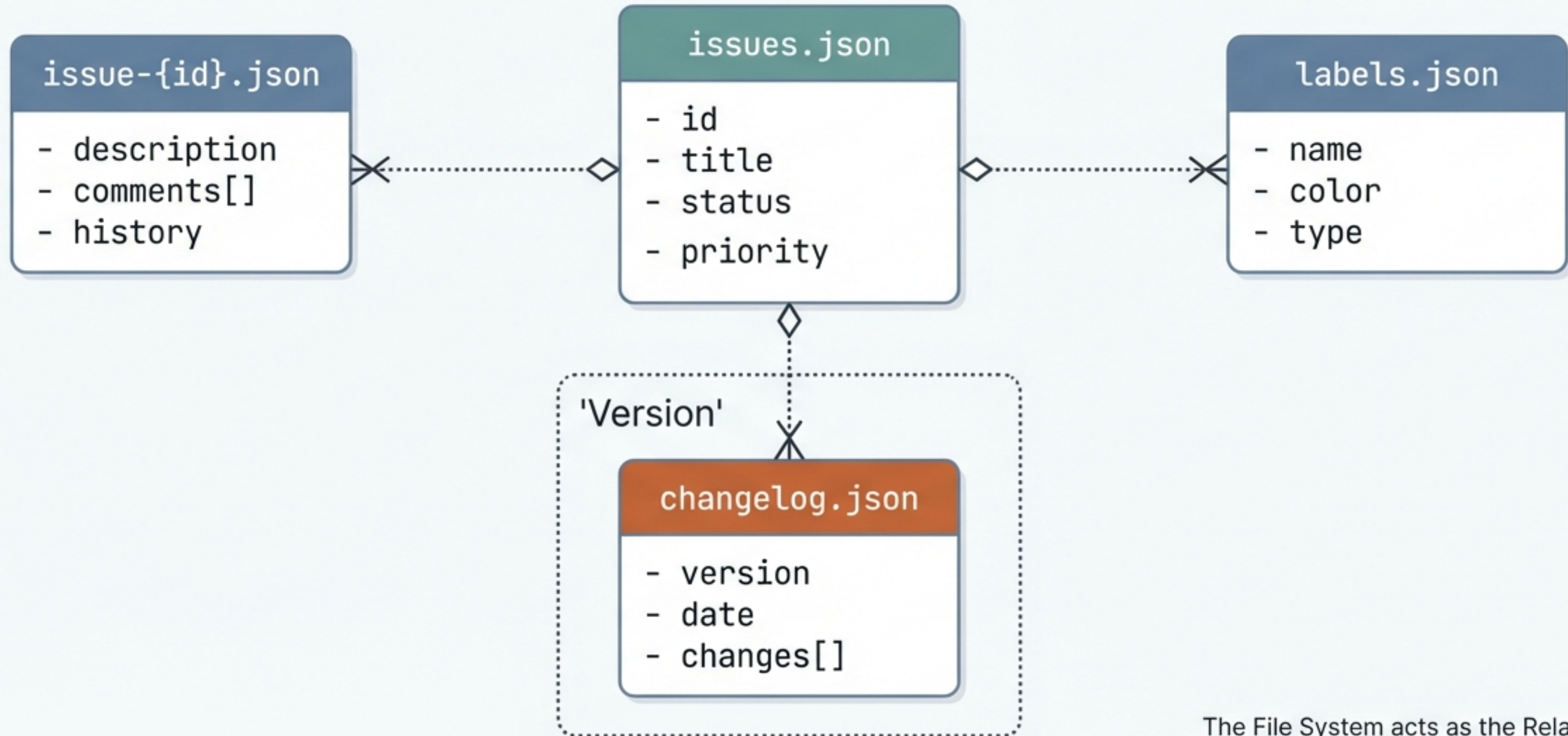
Every version folder is
self-contained. It houses its own
README.md, changelog.json, and
references to specific issues.

Structured Protocols



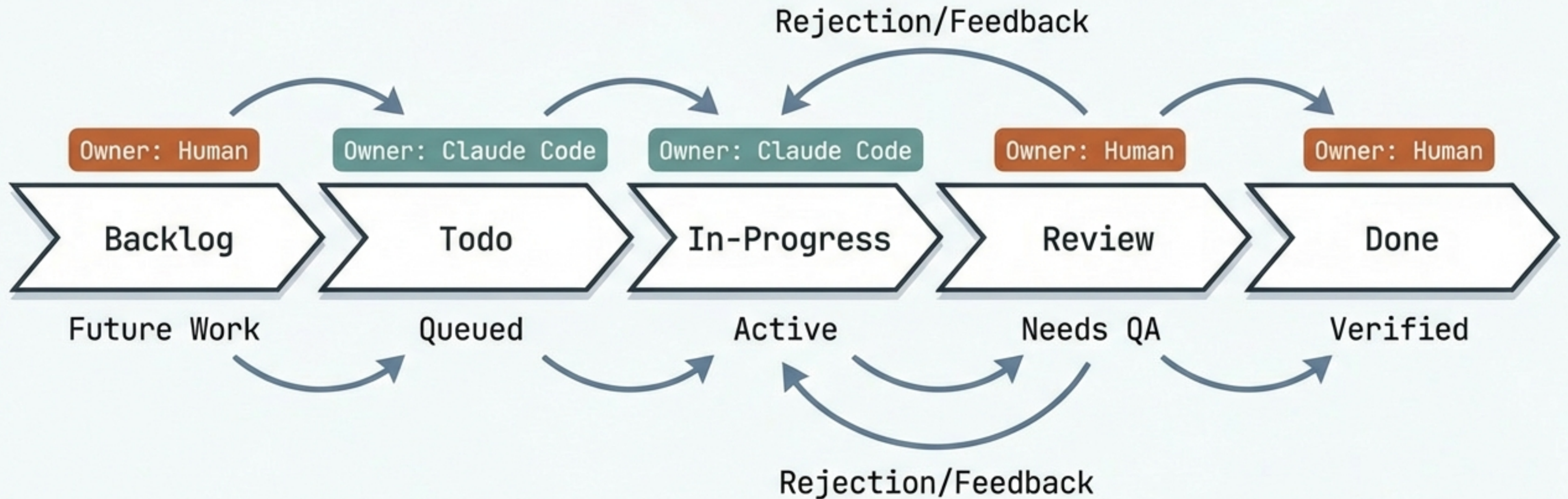
JSON schemas act as the
contract. They standardize how
work, issues, and status updates are
exchanged between Human and AI.

The Data Model: Schema Definitions

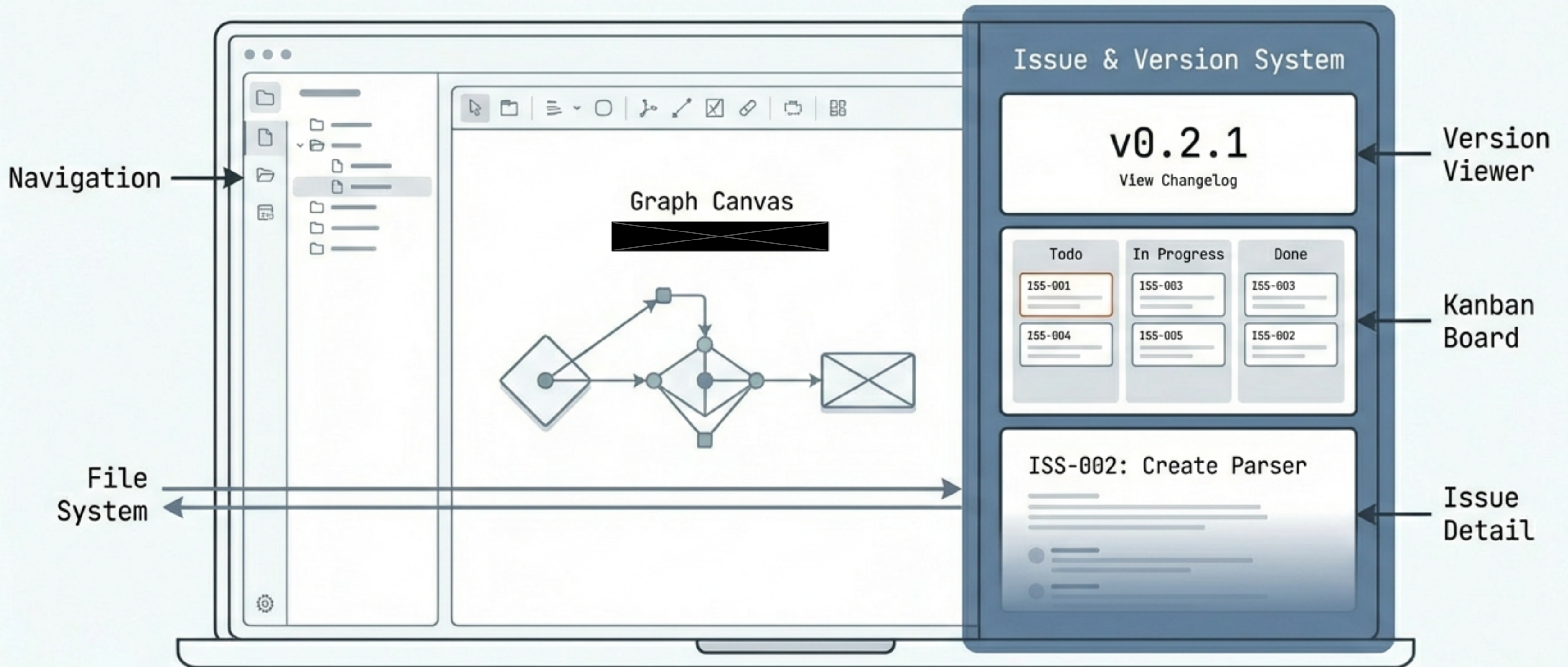


The File System acts as the Relational Store.

Workflow Logic: The Collaboration State Machine

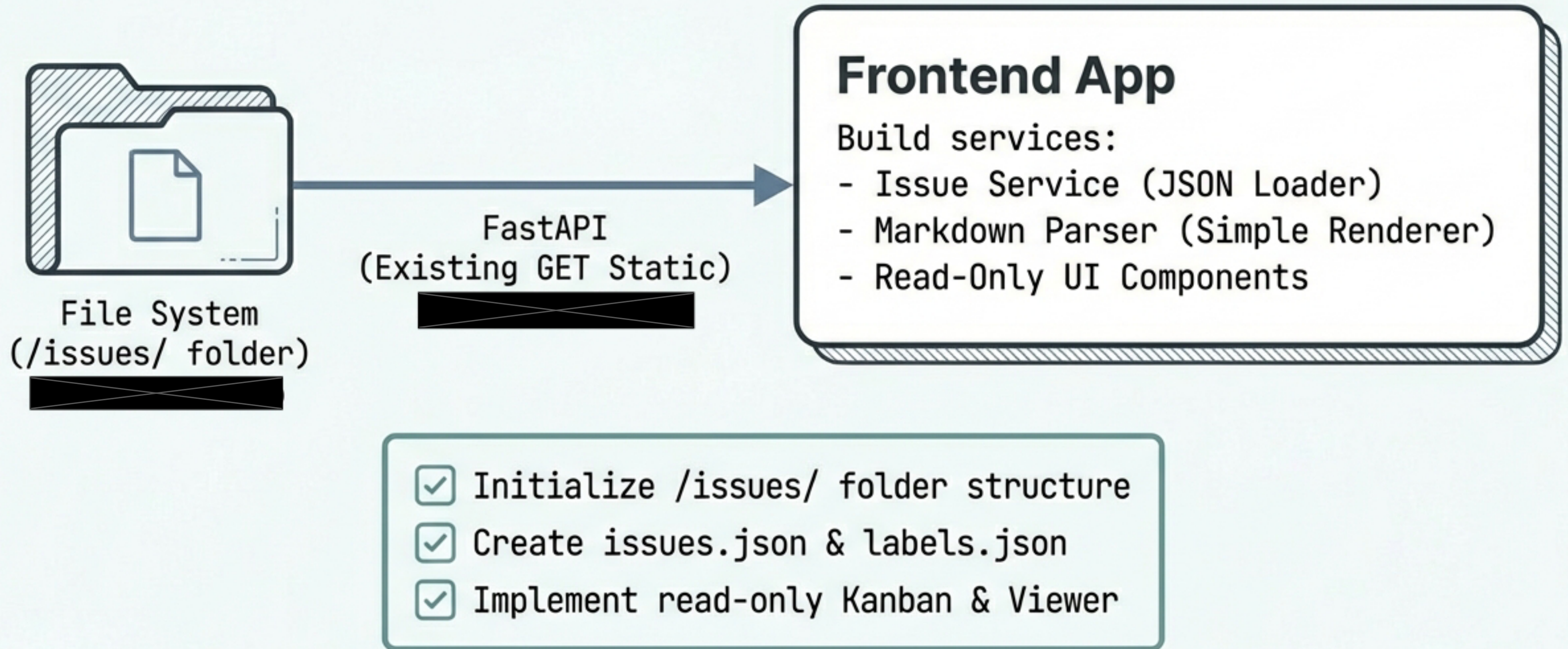


User Experience Components



Phase 1 Implementation: Frontend Focus

Goal: Low Friction Deployment. No Backend Changes.

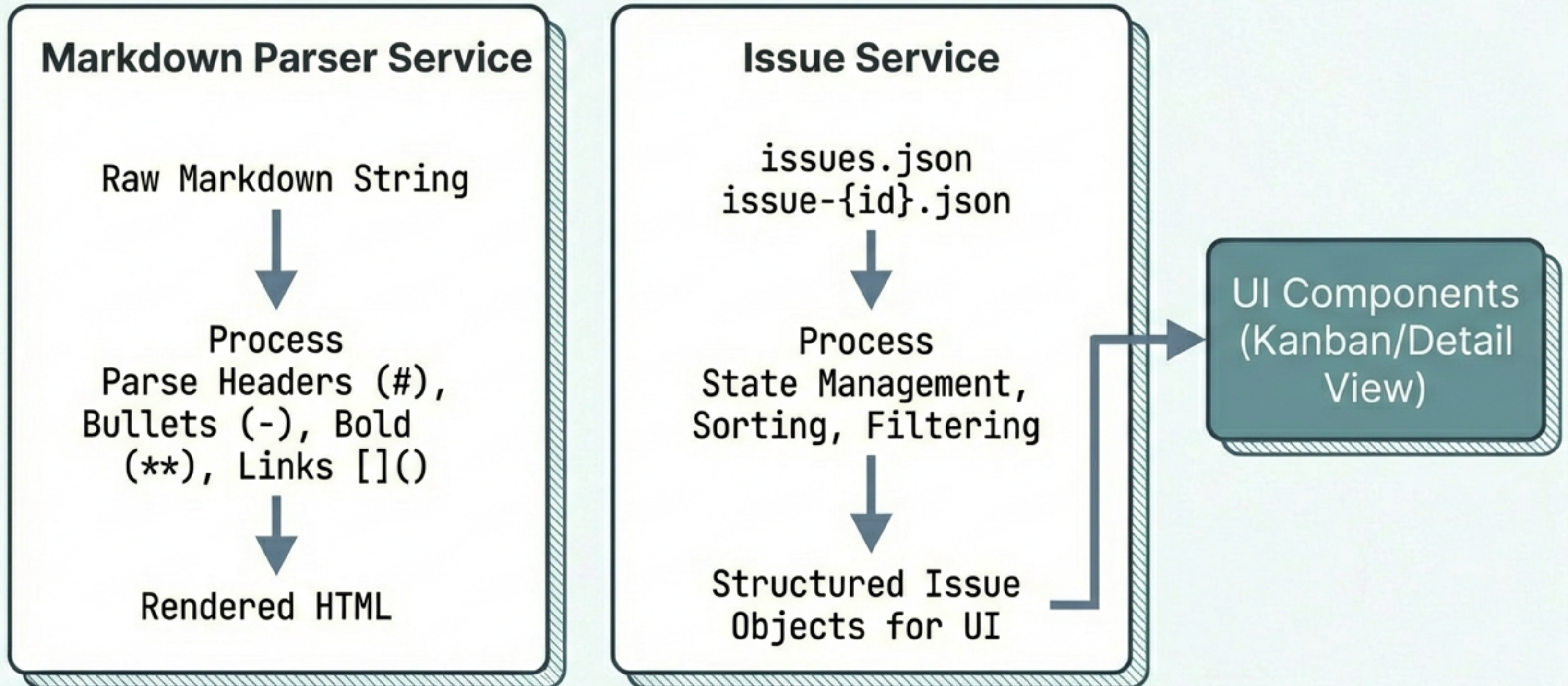


Phase 2 Implementation: Backend API

Goal: Full Interactivity. Requires FastAPI Changes.

Phase 1 (Read Only)	Phase 2 (Read/Write)
Data Access: Static File Serving 📁	Data Access: Dynamic CRUD 🗄️
Interactions: View, Filter, Sort	Interactions: Create, Update, Comment
Methods: GET only	Methods: • GET /api/issues (List) • POST /api/issues (Create) • PATCH /api/issues/{id} (Update Status) • POST /api/issues/{id}/comments (Discuss)

Service Architecture Requirements



Work Register: High Priority Targets

ISS-001 | Add README files to each release

[feature]

[documentation]

ISS-002 | Create markdown parser service

[feature]

[frontend]

ISS-003 | Implement Kanban board view

[feature]

[frontend]

ISS-004 | Create Issue Service

[feature]

[frontend]

Work Register: Medium & Low Priority

Medium Priority (UI Polish)

ISS-005 | Version Viewer
with README rendering

ISS-006 | Issue Detail panel

ISS-007 | Create changelog.
json for each version

Low Priority / Phase 2 (Backend)

ISS-008 | Backend API for
issue CRUD

ISS-009 | QA checklist
feature

ISS-010 | Version planning

Quality Assurance & Review Protocol

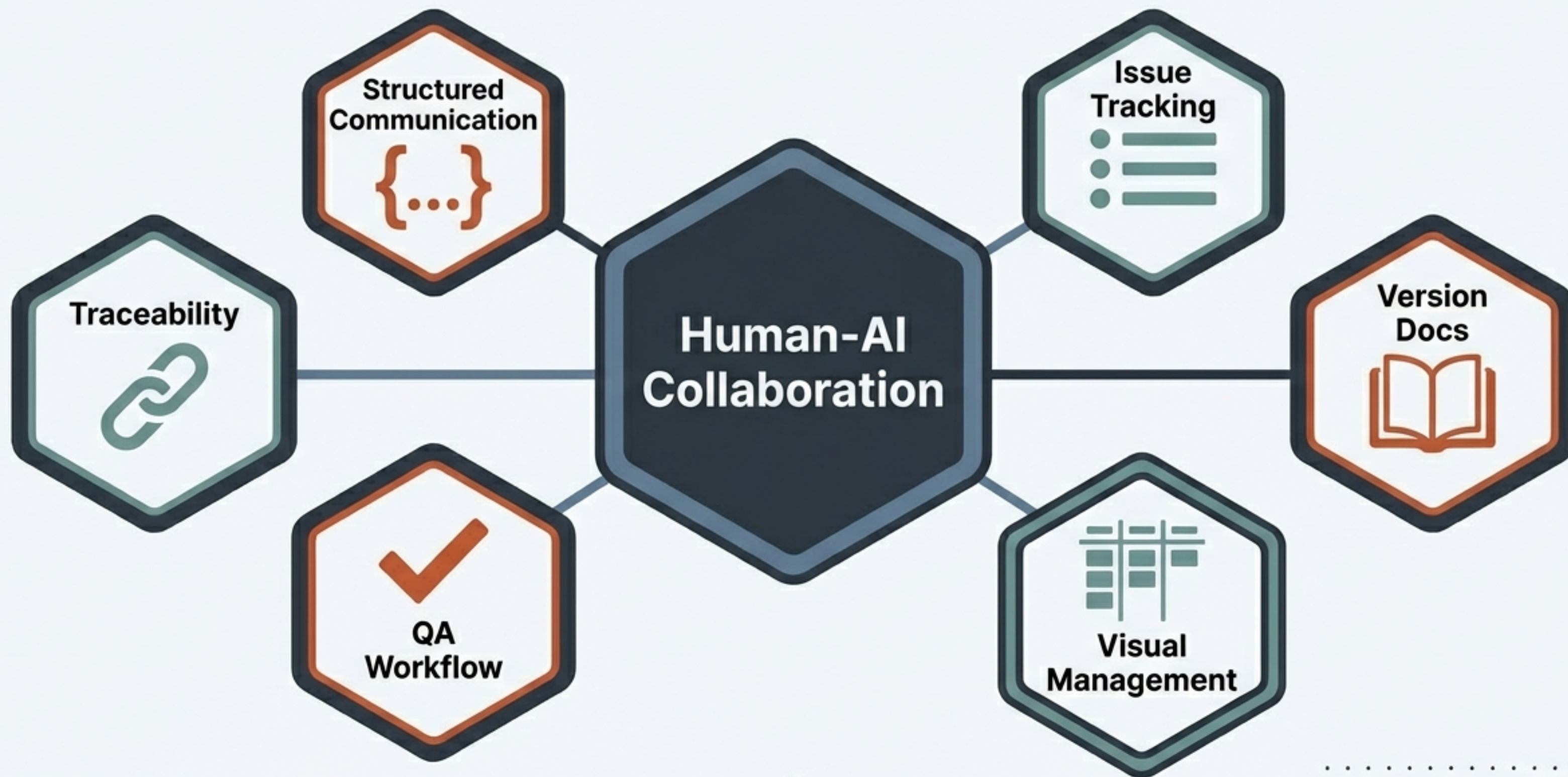


Review Protocol

- **1. Verify Schema Contract**
Does the JSON match the definition?
- **2. Changelog Integrity**
Is changelog.json updated?
- **3. Documentation Accuracy**
Does the README reflect the new code?
- **4. Transition Ownership**
Move status from 'Review' to 'Done'.



System Capabilities Summary



Immediate Execution Steps

```
$ mkdir /issues/  
$ touch /issues/issues.json  
$ touch /issues/labels.json
```

**Action 1:
Initialize Structure**



**Action 2: Begin ISS-002 (Markdown Parser)
& ISS-004 (Issue Service)**

Appendix: Status Definitions Reference

Status	Owner	Description
backlog	● Human	Future work, not yet prioritized.
todo	● Claude Code	Queued for implementation.
in-progress	● Claude Code	Currently being worked on.
review	● Human	Implementation complete, needs QA.
done	● Human	Verified and accepted.